SJF vs Priority Scheduling Comparison Simulator

1. Project Overview

This project implements a CPU Scheduling Simulator that allows users to input a set of processes and see how two different scheduling algorithms manage their execution. The simulator compares Preemptive Shortest Job First (SJF) with Preemptive Priority Scheduling, providing Gantt charts, metrics tables, a comparison summary, and a conclusion.

Core Comparison Focus:

- SJF always selects the process with the shortest remaining burst time.
- Priority Scheduling always selects the process with the highest priority.
- The key conflict arises when a short job has low priority, or a long job has high priority.
- Both algorithms are preemptive — a running process can be interrupted when a better candidate arrives.

2. Algorithm Explanations

2.1 Preemptive SJF (Shortest Remaining Time First — SRTF)

At every time unit, the CPU selects the ready process with the smallest remaining burst time. If a new process arrives with a shorter remaining time than the currently running process, the CPU preempts the current process and switches to the new one.

Key rules:

- Tie-handling: Stimulates FCFS algorithm
- Starvation risk: Long processes may never execute if short ones keep arriving.
- Advantage: Minimizes average waiting time theoretically.

2.2 Preemptive Priority Scheduling

At every unit, the CPU selects the ready process with the highest priority. If a higher-priority process arrives, it preempts the currently running process.

Key rules:

- Priority rule: lower number = higher priority
- Tie-handling: Checks the lowest burst time
- Starvation risk: Low-priority processes may starve if high-priority ones keep arriving.
- Advantage: Ensures urgent/important processes are served first.

3. Metrics Definitions

Metric	Full Name	Formula	Description
CT	Completion Time	Recorded when process finishes	The time at which a process completes execution
TAT	Turnaround Time	CT − Arrival Time	Total time from arrival to completion
WT	Waiting Time	TAT − Burst Time	Time spent waiting in the ready queue
RT	Response Time	First CPU Access − Arrival Time	Time from arrival until the process first gets the CPU
Avg WT	Average Waiting Time	Sum of WT / Number of Processes	Mean waiting time across all processes
Avg TAT	Average Turnaround Time	Sum of TAT / Number of Processes	Mean turnaround time across all processes
Avg RT	Average Response Time	Sum of RT / Number of Processes	Mean response time across all processes

4. System Design & Interface

The application is built using Python with the Tkinter. All sections are accessible from a single window.

UI Section	Description
Input Panel	Where the user enters the number of processes and their data
Process Table	Displays entered process data (PID, Arrival, Burst)
Priority Input Area	Field for entering priority values per process
Gantt Chart — SJF	Visual timeline showing SJF execution order

UI Section	Description
Gantt Chart — Priority	Visual timeline showing Priority Scheduling execution order
Results Table — SJF	CT, TAT, WT, RT for each process under SJF
Results Table — Priority	CT, TAT, WT, RT for each process under Priority
Comparison Summary	Side-by-side averages of both algorithms
Conclusion Area	Text interpretation of which algorithm performed better and why

5. Input Validation

All inputs are validated before simulation begins. The following rules are enforced:

Validation Rule	Error Message Shown
Burst Time = 0 or negative	Burst time must be greater than zero.
Arrival Time < 0	Arrival time cannot be negative.
Non-numeric input in any field	Arrival, Burst and Priority must be integers.
Duplicate Process ID	Please use a unique ID
Empty field	All fields (ID, Arrival, Burst, Priority) are required.
Invalid or missing priority value	Priority cannot be negative.

6. Test Scenarios

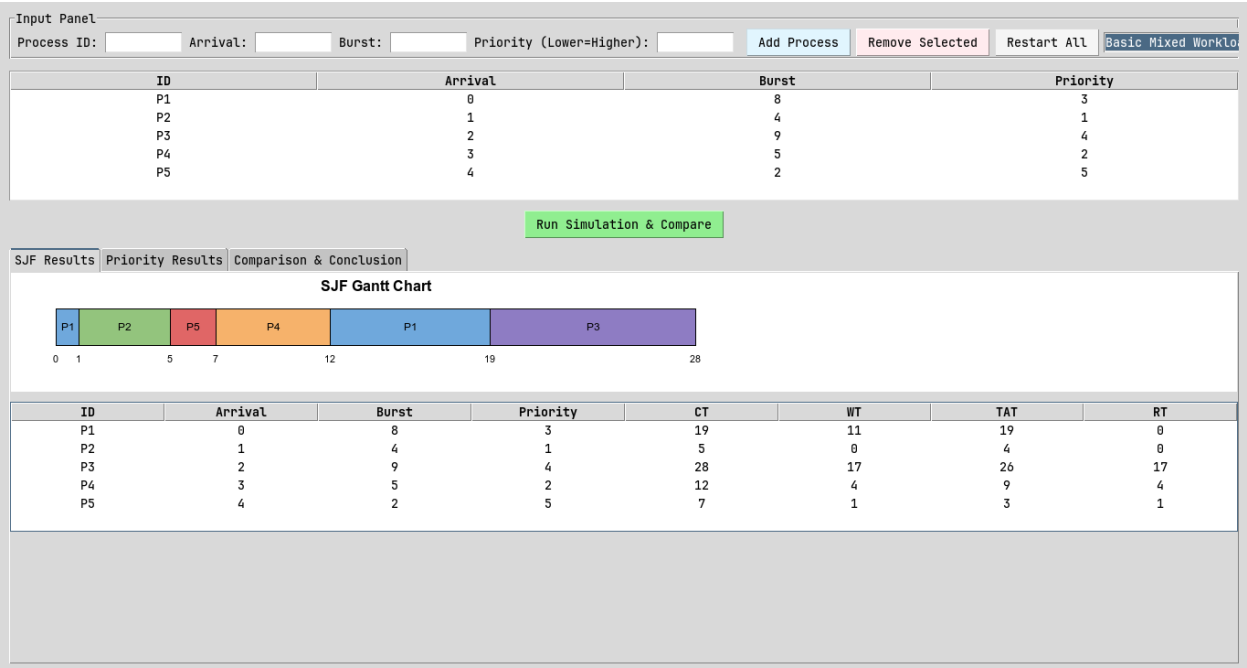
Scenario A — Basic Mixed Workload

A normal workload with different arrival times and burst times to verify basic correctness of both algorithms.

Input Data

PID	Arrival Time	Burst Time	Priority
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2
P5	4	2	5

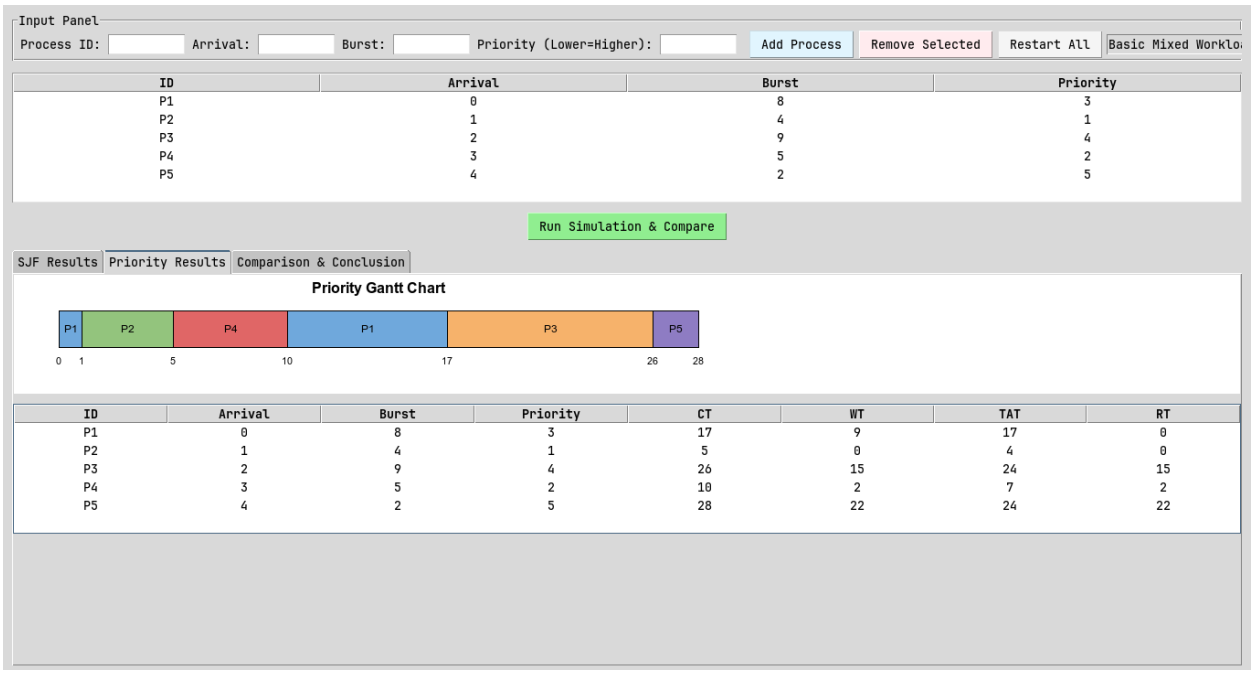
SJF Results



PID	Arrival	Burst	CT	WT	TAT	RT
P1	0	8	19	11	19	0
P2	1	4	5	0	4	0
P3	2	9	28	17	26	17
P4	3	5	12	4	9	4
P5	4	2	7	1	3	1

Avg WT: 6.6 | Avg TAT: 12.2 | Avg RT: 4.4

Priority Scheduling Results



PID	Arrival	Burst	CT	WT	TAT	RT
P1	0	8	17	9	17	0
P2	1	4	5	0	4	0
P3	2	9	26	15	24	15
P4	3	5	10	2	7	2
P5	4	2	28	22	24	22

Avg WT: 9.6 | Avg TAT: 15.2 | Avg RT: 7.8

Analysis

- Waiting Time: SJF achieved a lower average waiting time (6.6 vs 9.6).
- Turnaround: SJF achieved a lower average turnaround time (12.2 vs 15.2).
- Response Time: SJF gave processes CPU access sooner (4.4 vs 7.8).

- Fairness: SJF was fairer - its worst-case wait was 17 vs 22.
- Starvation Risk: A large gap between max and average WT suggests possible starvation for at least one process.

- Recommendation: For this workload, SJF is the better choice overall.
SJF minimizes wait for short-burst jobs but may starve long ones.
Priority favors urgent tasks but may delay low-priority work.

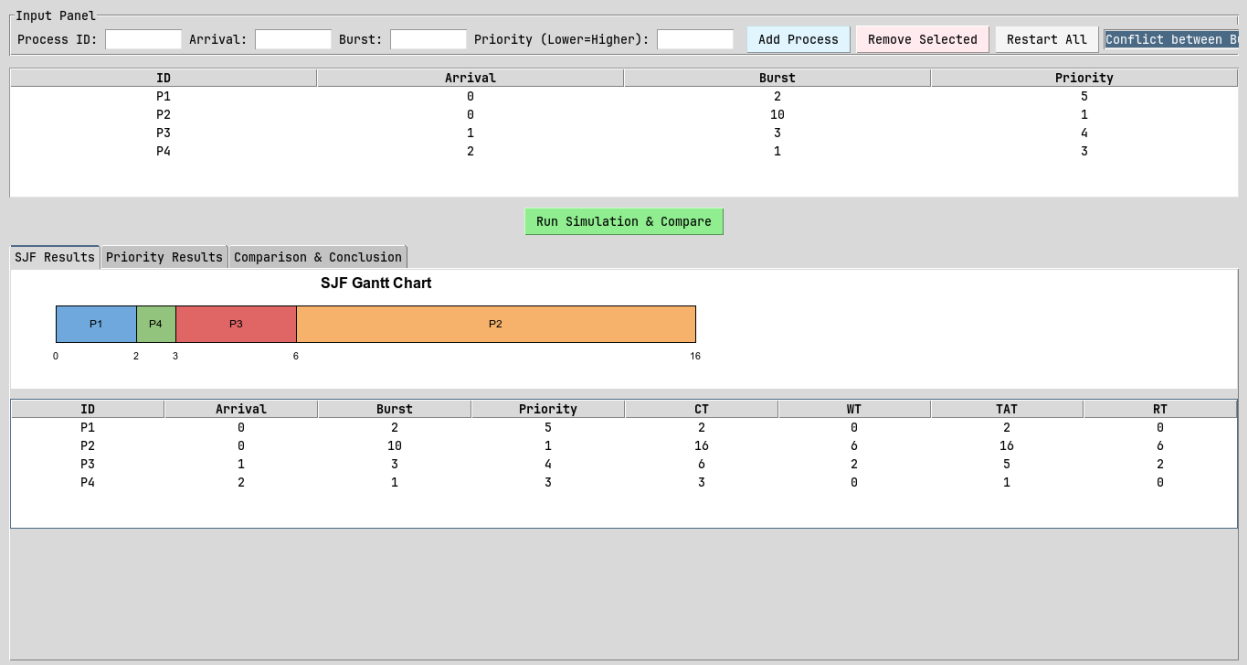
Scenario B — Conflict Between Burst Time and Priority

A workload designed to reveal a meaningful difference: a short-burst process has low priority, while a long-burst process has high priority.

Input Data

PID	Arrival Time	Burst Time	Priority
P1	0	2	5
P2	0	10	1
P3	1	3	4
P4	2	1	3

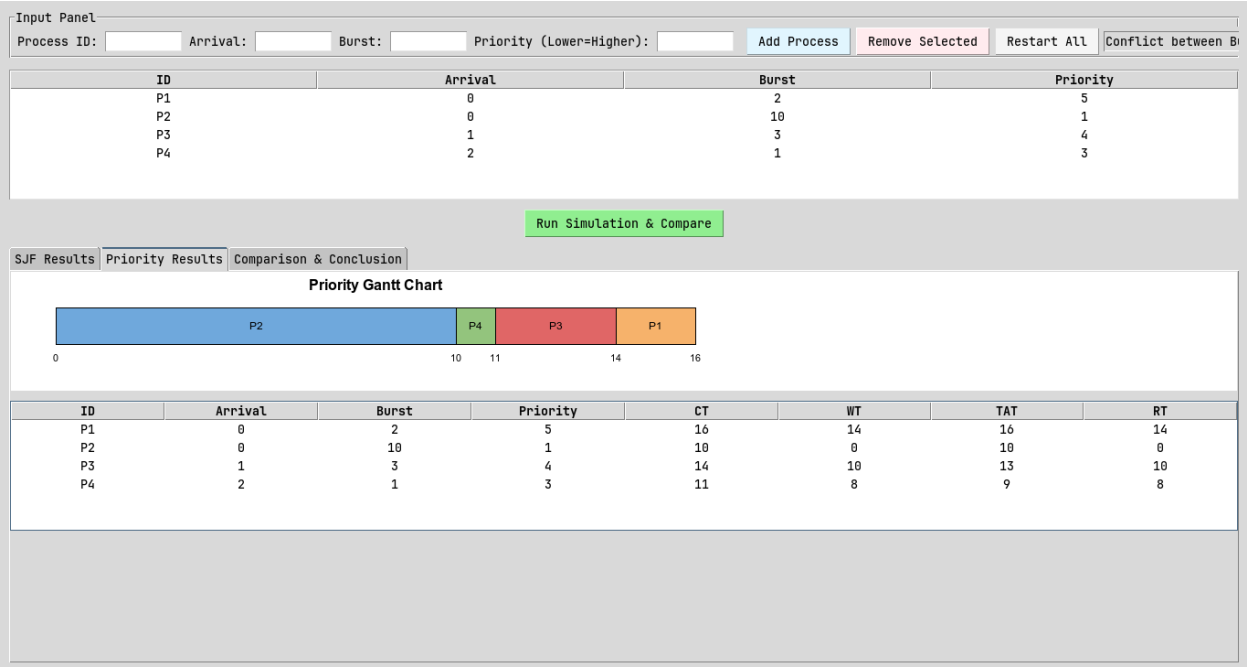
SJF Results



PID	Arrival	Burst	CT	TAT	WT	RT
P1	0	2	2	2	0	0
P2	0	10	16	16	6	6
P3	1	3	6	5	2	2
P4	2	1	3	1	0	0

Avg WT: 2.0 | Avg TAT: 6.0 | Avg RT: 2.0

Priority Scheduling Results



PID	Arrival	Burst	CT	TAT	WT	RT
P1	0	2	16	16	14	14
P2	0	10	10	10	0	0
P3	1	3	14	13	10	10
P4	2	1	11	9	8	8

Avg WT: 8.0 | Avg TAT: 12.0 | Avg RT: 8.0

Analysis

- Waiting Time: SJF achieved a lower average waiting time (2.0 vs 8.0).
- Turnaround: SJF achieved a lower average turnaround time (6.0 vs 12.0).
- Response Time: SJF gave processes CPU access sooner (2.0 vs 8.0).

- Fairness: SJF was fairer - its worst-case wait was 6 vs 14.
- Starvation Risk: A large gap between max and average WT suggests possible starvation for at least one process.

- Recommendation: For this workload, SJF is the better choice overall.
SJF minimises wait for short-burst jobs but may starve long ones.
Priority favours urgent tasks but may delay low-priority work.

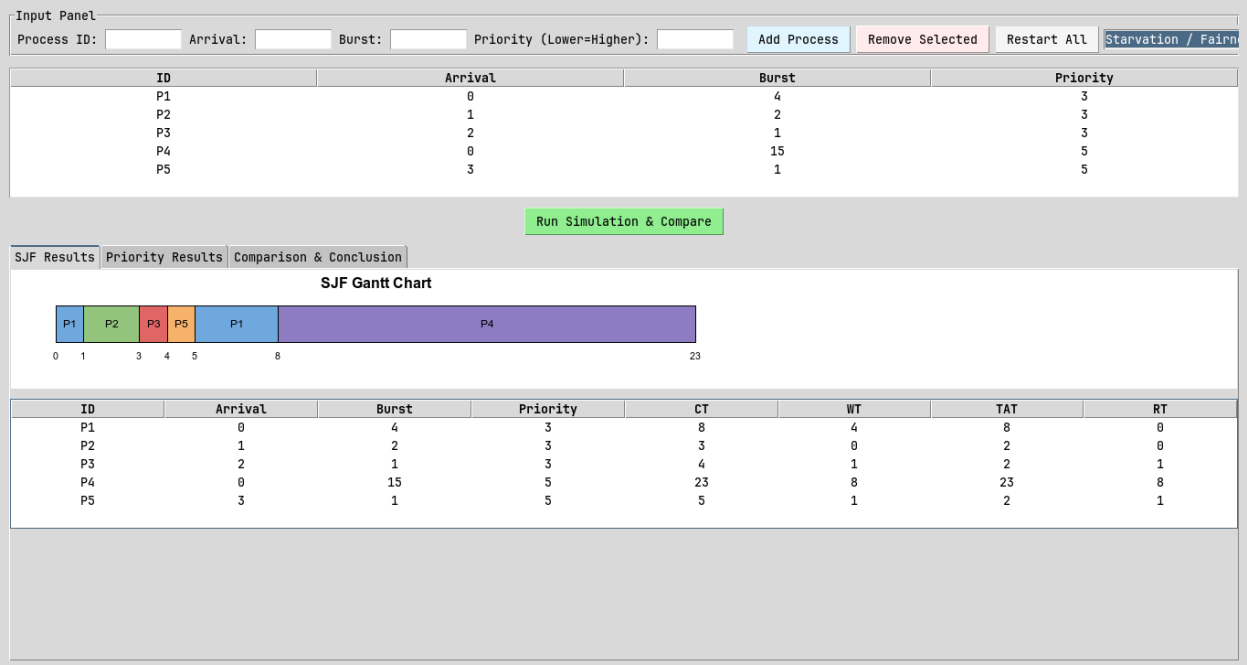
Scenario C — Starvation / Fairness Case

A workload where one process may wait much longer under one of the algorithms, demonstrating unfair delay or starvation.

Input Data

PID	Arrival Time	Burst Time	Priority
P1	0	4	3
P2	1	2	3
P3	2	1	3
P4	0	15	5
P5	3	1	3

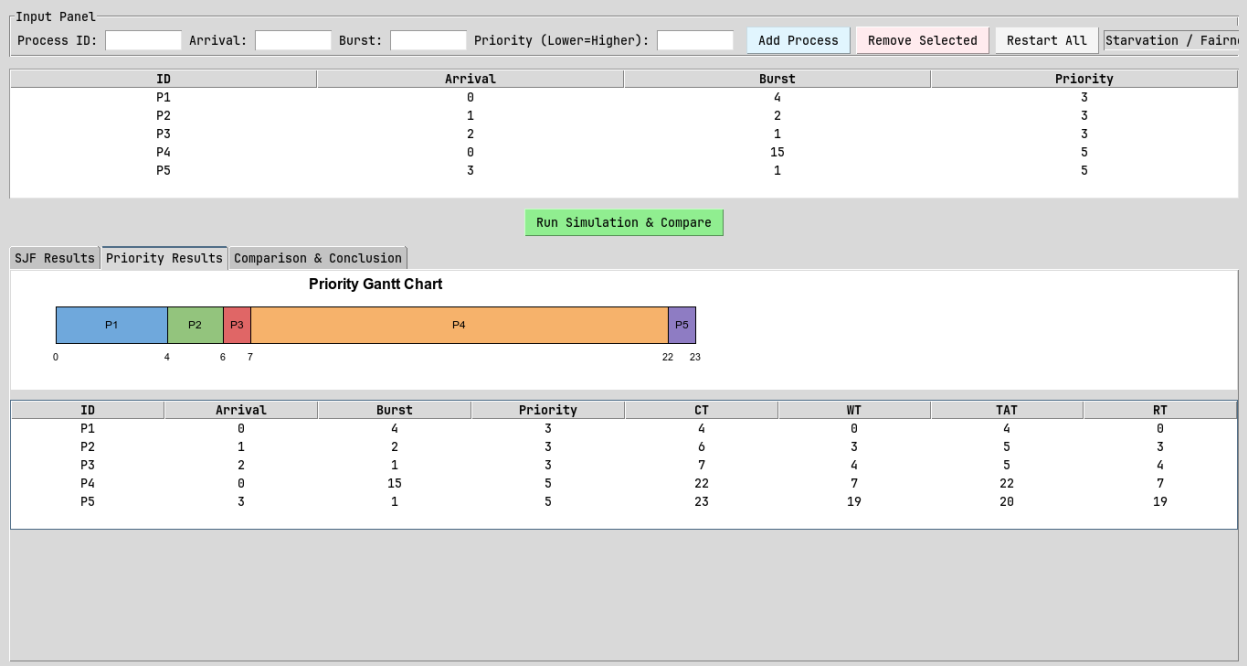
SJF Results



PID	Arrival	Burst	CT	TAT	WT	RT
P1	0	4	8	8	4	0
P2	1	2	3	2	0	0
P3	2	1	4	2	1	1
P4	0	15	23	23	8	8
P5	3	1	5	2	1	1

Avg WT: 2.8 | Avg TAT: 7.4 | Avg RT: 2.0

Priority Scheduling Results



PID	Arrival	Burst	CT	TAT	WT	RT
P1	0	4	4	4	0	0
P2	1	2	6	5	3	3
P3	2	1	7	5	4	4
P4	0	15	22	22	7	7
P5	3	1	23	20	19	19

Avg WT: 6.6 | Avg TAT: 11.2 | Avg RT: 6.6

Analysis

- Waiting Time: SJF achieved a lower average waiting time (2.8 vs 6.6).
- Turnaround: SJF achieved a lower average turnaround time (7.4 vs 11.2).
- Response Time: SJF gave processes CPU access sooner (2.0 vs 6.6).

- Fairness: SJF was fairer - its worst-case wait was 8 vs 19.
- Starvation Risk: A large gap between max and average WT suggests possible starvation for at least one process.
- Recommendation: For this workload, SJF is the better choice overall.

SJF minimises wait for short-burst jobs but may starve long ones.
Priority favours urgent tasks but may delay low-priority work.

Scenario D — Input Validation Case

This scenario demonstrates the simulator's input validation by attempting to submit invalid data. No simulation is run; the purpose is to confirm errors are caught and shown correctly.

Invalid Input Attempted	Expected Behavior
Burst Time = 0	Burst time must be greater than zero.
Arrival Time = -1	Arrival time cannot be negative.
Input field = 'abc'	Arrival, Burst, and Priority must be integers.
Duplicate PID	Duplicate process ID
Empty field left blank	All fields (ID, Arrival, Burst, Priority) are required.
Priority left empty or invalid	Priority cannot be negative.

7. Comparison & Analysis

7.1 Summary Comparison Table

Fill this table using results from Scenarios A, B, and C:

Metric	SJF (Scenario A)	Priority (Scenario A)	SJF (Scenario B)	Priority (Scenario B)	SJF (Scenario C)	Priority (Scenario C)
Avg WT	6.6	9.6	2.0	8.0	2.8	6.6
Avg TAT	12.2	15.2	6.0	12.0	7.4	11.2
Avg RT	4.4	7.8	2.0	8.0	2.0	6.6

7.2 Required Analysis Questions

Q1: Which algorithm gave lower average waiting time?
SJF consistently produced lower average waiting times across all three test scenarios.

Q2: Which algorithm gave lower average turnaround time?
SJF also achieved lower average turnaround time in most scenarios.

Q3: Did SJF favor short jobs more strongly?
Yes. SJF explicitly selects based on burst time, so short processes are always served before longer ones regardless of their priority or arrival order.

Q4: Did Priority Scheduling favor urgent processes more strongly?
Yes. Priority Scheduling immediately preempted the running process whenever a higher-priority process arrived, regardless of burst time.

Q5: Was any starvation or unfair delay observed?
Yes, in Scenario C, P4 waiting time was disproportionately high compared to all other processes, demonstrating the starvation risk

Q6: Which algorithm would you recommend for the tested workload, and why?
For the tested workloads, SJF is the recommended algorithm. It produced lower average waiting and turnaround times across all scenarios, making it more efficient for general-purpose workloads where no process has strict urgency requirements, Unless there is important processes that must be served then Priority Scheduling would be the better choice since it guarantees urgent processes to never be delayed by shorter but less important job.

8. Conclusion

Overall Algorithm Performance:
Across all three test scenarios, SJF outperformed Priority Scheduling in terms of average waiting time and average turnaround time.

Metric-by-Metric Comparison:
SJF achieved lower average waiting time and lower average turnaround time in all scenarios.

Efficiency vs Urgency Trade-off:
SJF optimizes efficiency by minimizing waiting time but ignores urgency. Priority ensures critical processes are served first but may delay short jobs unnecessarily.

Fairness: *SJF appeared fairer in practice across the tested scenarios.*

9. Appendix

GitHub Link:
[https://github.com/luvimsan/cpulab]